



Paralelización con OpenMP

Ecuación de Poisson en 2D

Sistemas Distribuidos — Análisis de Escalabilidad HPC

Camilo Huertas Sebastián Rodríguez

Programa Académico de Física

Universidad Distrital Francisco José de Caldas

Docente: Carlos Andrés Gómez Vasco | 24 de marzo de 2026

Contenido

- 1 Objetivo y Sistema HPC
- 2 Código Base Secuencial
- 3 Análisis de Escalabilidad
- 4 Actividad 1 — `parallel for`
- 5 Actividad 2 — `collapse(2)`
- 6 Actividad 3 — `sections`
- 7 Actividad 4 — `schedule`
- 8 Actividad 5 — Sincronización Explícita
- 9 Actividad 6 — `critical` vs `atomic`
- 10 Actividad 7 — `task`
- 11 Comparación Final
- 12 Validación Numérica
- 13 Conclusiones

Objetivo del Proyecto

Objetivo

Aplicar directivas de **OpenMP** para paralelizar la **Ecuación de Poisson 2D** (diferencias finitas). Se evaluará la *escalabilidad*, cuellos de botella de memoria y límites de aceleración (Ley de Amdahl) mediante un barrido paramétrico masivo.

Problema Físico

La ecuación de Poisson en 2D:

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = f(x, y)$$

- Término fuente $f(x, y) = 4$.
- Esquema centrado *in-place*.
- Tolerancia iterativa $TOL = 10^{-6}$.

Directivas Evaluadas

- `parallel for + reduction`
- `collapse(2)`
- `sections`
- `schedule(static)`
- `nowait, barrier, single, critical`
- `atomic` | `task`

Especificaciones del Sistema HPC

Hardware del Servidor

Componente	Especificación
Procesador	AMD Ryzen Threadripper 3990X
Arquitectura	x86_64
Núcleos / Hilos	64 Físicos / 128 Lógicos
Caché L3	256 MiB

Relevancia Arquitectónica

La caché L3 de 256 MiB es clave: al superarse, el ancho de banda a la RAM se vuelve el **cuello de botella dominante** (Von Neumann).

Metodología de Pruebas

- Mallas: 50^2 , 100^2 , 200^2 (hasta 512^2)
- Hilos:
 $p \in \{1, 2, 4, 8, 16, 32, 64, 80\}$

Sweet Spot Identificado

El máximo rendimiento se alcanza con **32 hilos**. Por encima de 64, el *overhead* destruye el *speedup*.

Código Base: Línea de Referencia

Discretización (Diferencias Finitas)

$$T_{i,j}^{n+1} = \frac{(T_{i+1,j} + T_{i-1,j})k^2 + (T_{i,j+1} + T_{i,j-1})h^2 - f_{i,j}h^2k^2}{2(h^2 + k^2)}$$

```

1 while (delta > TOL) {
2     delta = 0.0;
3     for (int i = 1; i < M; ++i)
4         for (int j = 1; j < N; ++j) {
5             double T_old = T[i][j];
6             T[i][j] = (((T[i+1][j]+T[i-1][j])*k*k)
7                 + ((T[i][j+1]+T[i][j-1])*h*h)
8                 - (source[i][j]*h*h*k*k))
9                 / (2.0*(h*h+k*k));
10            double diff = abs(T[i][j]-T_old);
11            if (diff > delta) delta = diff;
12        }
13        iterations++;
14    }

```

Listing 1: Bucle iterativo secuencial

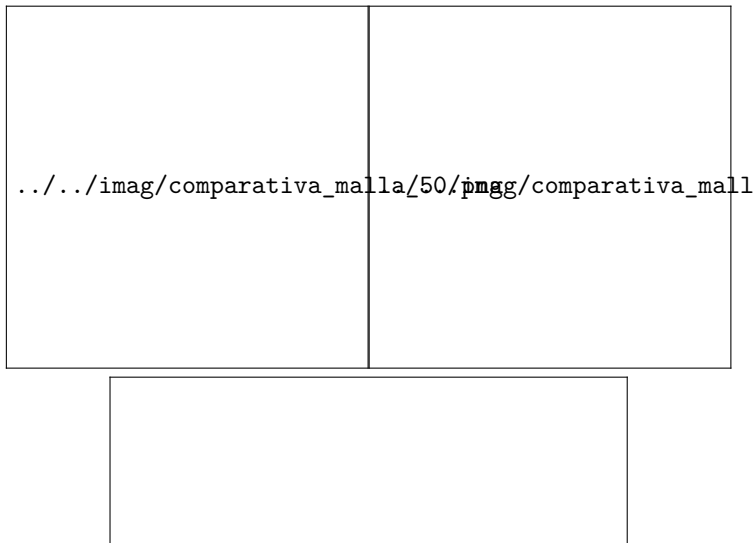
Resultados (1 hilo)

Malla	Tiempo (s)	Iter.
64 ²	0.088	3 136
128 ²	1.220	10 244
256 ²	15.404	31 844
512 ²	177.864	91 868

Observación

Las iteraciones crecen como $\mathcal{O}(N^2)$.
Convergencia lenta típica de Gauss-Seidel.

Curvas de Rendimiento — Barrido Paramétrico



Hallazgos Clave

- El *sweet spot* universal es **32 hilos**.
- Por encima de **64 hilos**: inflexión al alza.
- Causa: saturación del ancho de banda de la memoria compartida (*Von Neumann Bottleneck*).

Act. 1 — #pragma omp parallel for

```

1 #pragma omp parallel for reduction(max:delta)
2 for (int i = 1; i < M; ++i) {
3     for (int j = 1; j < N; ++j) {
4         double T_old = T[i][j];
5         T[i][j] = (((T[i+1][j]+T[i-1][j])*k*k) + ((T[i][j+1]+T[i][j-1])*h*h)
6                 -(source[i][j]*h*h*k*k)) / (2.0*(h*h+k*k));
7         double diff = abs(T[i][j]-T_old);
8         if (diff > delta) delta = diff;
9     }
10 }
11 iterations++;

```

Listing 2: Paralelización del bucle principal

¿Por qué reduction(max:delta)?

- Evita *data races* al sobrescribir delta.
- Garantiza un máximo local por hilo, combinado de forma segura al final.

Speedup en Malla 512²

Hilos	Tiempo (s)	Speedup
1	177.86	1.00×
4	44.77	3.97×
16	11.81	15.06×
32	6.63	26.82×
64	5.50	32.36×

Veredicto Arquitectónico

Mejor costo-beneficio: 32× de *speedup* con una sola línea de código.

Para 100 % de determinismo numérico, combinar con métodos de *Jacobi* o *Red-Black*.

Act. 2 — collapse(2): Fusión de Bucles

```

1 #pragma omp parallel for collapse(2) reduction(max:delta)
2 for (int i = 1; i < M; ++i) {
3     for (int j = 1; j < N; ++j) {
4         // ... diferencias finitas ...
5     }
6 }

```

Listing 3: Espacio de iteración fusionado

¿Qué hace collapse(2)?

Fusiona bucles en un espacio lineal único de $(M-1) \times (N-1)$ iteraciones, aumentando la granularidad para el *scheduler*.

Comparativa vs parallel for (512², 64 hilos)

Variante	Tiempo (s)	Speedup
parallel for	5.50	32.36×
collapse(2)	5.83	30.52×

¿Mejóro? No.

Introduce *overhead* por el recálculo de índices. En mallas cuadradas donde $M \gg p$, el bucle externo ya es suficiente.

¿Cuándo sí usarlo?

Solo cuando el bucle externo tiene muy pocas iteraciones ($M \approx p$).

Act. 3 — sections: Inicialización en Paralelo

```

1 // Memoria reservada ANTES de la región paralela
2 std::vector<std::vector<double>> T(M+1, std::vector<double>(N+1, 0.0))
3 ;
4 std::vector<std::vector<double>> source(M+1, std::vector<double>(N+1,
5 0.0));
6
7 #pragma omp parallel sections
8 {
9     #pragma omp section
10     initialize_boundaries(M, N, T, h, k);
11
12     #pragma omp section
13     poisson_source(M, N, source);
14 }

```

Listing 4: Bifurcación de inicialización

Speedup Malla 512^2 (Total)

Hilos	Tiempo (s)	Speedup
1	177.86	1.00×
16	14.06	12.65×
64	6.03	29.50×

¿Tuviste que reordenar? Sí

Para evitar *Seg-Faults*, la memoria debe asignarse estáticamente en el hilo maestro.

¿Vale la pena? No

- Inicialización $\mathcal{O}(N^2)$ es trivial.
- El *overhead* de fork-join excede el beneficio.
- Ideal solo para I/O asimétrico.

Act. 4 — Control Explícito y schedule

```

1 #pragma omp parallel // Fork explícito
2 {
3     #pragma omp for schedule(static) reduction(max:delta)
4     for (int i = 1; i < M; ++i) {
5         for (int j = 1; j < N; ++j) {
6             // Discretización de Poisson
7         }
8     }
9 } // Join implícito

```

Listing 5: Región paralela separada del bucle

static vs dynamic

- **static:** Reparto en compilación. Ideal para FLOPs uniformes (como Poisson).
- **dynamic:** Reparto en ejecución. Ideal para cargas desbalanceadas.

Speedup Malla 512² — schedule(static)

Hilos	Tiempo (s)	Speedup
1	177.86	1.00×
16	11.78	15.10×
32	6.72	26.46×
64	5.55	32.06×

Por qué NO se usó dynamic

Cada celda ejecuta la misma carga. Forzar asignación dinámica inunda el bus con *overhead* de control.

Ventaja parallel + for

No recrear el equipo de hilos en cada iteración del while reduce latencia acumulada.

Act. 5 — `nowait`, `barrier`, `single`, `critical`

```

1 double local_delta = 0.0;
2
3 #pragma omp for nowait
4 for (int i = 1; i < M; ++i)
5     for (int j = 1; j < N; ++j) {
6         /* calc diff */
7         if (diff > local_delta) local_delta = diff;
8     }
9
10 #pragma omp critical
11 { if (local_delta > delta) delta = local_delta; }
12
13 #pragma omp barrier
14
15 #pragma omp single
16 { iterations++; }

```

Listing 6: Sincronización manual

Resultados — Malla 512²

Hilos	Tiempo (s)	Speedup
1	177.86	1.00×
16	12.24	14.52×
64	7.13	24.94×

`nowait`: ¿hubo cambio?

El beneficio fue anulado por el embotellamiento del `critical` y la `barrier` posterior. Peor que `reduction`.

Roles Clave

- **single**: Evita que `iterations` sume p veces por barrido.
- **critical**: Solo para estructuras complejas no soportadas por hardware.

Act. 6 — critical vs atomic: Software vs Hardware

```

1 // Estrategia 1: Software
2 #pragma omp critical(update_delta)
3 {
4     if (local_delta > delta) delta = local_delta;
5 }
6
7 // Estrategia 2: Hardware
8 #pragma omp atomic
9 iterations++;

```

Listing 7: Sincronización escalares

Alta Concurrency (64 hilos)

Malla	critical (s)	atomic (s)	Mejora
64 ²	0.169	0.092	+45 %
128 ²	0.479	0.278	+41 %
256 ²	1.946	1.208	+38 %
512 ²	7.847	6.605	+15 %

Mecanismo Interno

- **critical**: Mutexes del S.O. Genera *context switch* (costoso).
- **atomic**: Instrucciones nativas en chip (Ej. LOCK INC). Ejecuta en **~3-5 ciclos**.

Regla General

atomic siempre para contadores/sumas simples.
critical solo para bloques multilínea o memoria dispersa.

Nota Arquitectónica

Usar atomic dentro de single es redundante. Brilla cuando hay concurrencia real sin control único.

Act. 7 — Paralelismo de Tareas (task)

```

1  const int BS = 16; // Block Size
2  #pragma omp parallel shared(T,source,delta)
3  {
4      #pragma omp single
5      {
6          for (int bi=1; bi<M; bi+=BS)
7              for (int bj=1; bj<N; bj+=BS)
8                  #pragma omp task firstprivate(bi,bj) shared(T,delta)
9                  {
10                     double loc_d = 0.0;
11                     for (int i=bi; i<min(bi+BS,M); i++)
12                         for (int j=bj; j<min(bj+BS,N); j++)
13                             { /* calc diff */ }
14
15                     #pragma omp critical
16                     if (loc_d > delta) delta = loc_d;
17                 }
18             } // taskwait implícito
19             iterations++;
20 }

```

Listing 8: División por bloques

Colapso de Rendimiento (512²)

Hilos	Tiempo (s)	Speedup
4	124.3	1.43×
32	319.4	0.55×
64	589.7	0.30×
parallel for base: 5.50 s (32.36×		

¿Por qué es $\approx 100\times$ más lento?

Bloques 16×16 generan **1024** tareas **por iteración temporal**. El *scheduler* colapsa encolando y despachando, no calculando.

¿Cuándo sí usar task?

Estructuras irregulares: grafos, árboles, recursividad.
Matrices densas $\rightarrow$ parallel for.

Tabla Comparativa — Malla 512×512 , 64 Hilos

Versión	Directiva	Tiempo	Speedup	Diagnóstico HPC
Secuencial	N/A	177.86 s	1.00×	Baseline de normalización.
Parallel For	<code>parallel for</code>	5.48 s	32.5×	Superior. Balance óptimo.
Schedule	<code>schedule</code>	5.52 s	32.1×	Equivalente al básico. Estático ideal.
Collapse	<code>collapse(2)</code>	5.78 s	30.7×	Leve penalización por índices lineales.
Sections	<code>sections</code>	6.00 s	29.6×	Fork-join no justificado para $O(N^2)$.
Atomic	<code>atomic</code>	6.61 s	26.9×	Sincronización eficiente en hardware.
Critical	<code>critical</code>	7.13 s	24.9×	Penalización por <i>lock contention</i> .
Tareas	<code>task</code>	117.5 s	1.51×	Colapso. Planificador asfixiado.

Regla de Oro

Para EDPs en dominios regulares:
parallel for + reduction $\gg$ todo lo demás.

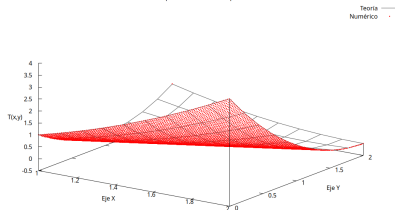
Trampa Frecuente

Más hilos $\neq$ más *speedup*. El *overhead* domina si la carga por hilo es minúscula.

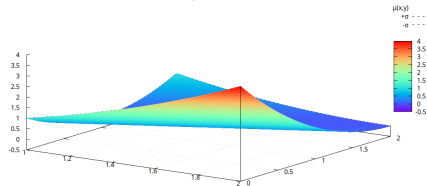
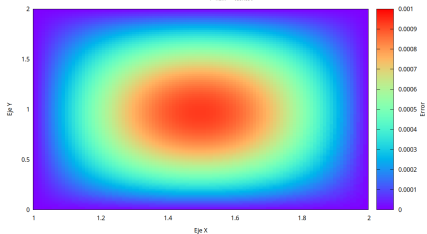
Validación Numérica de la Solución

Validación de Rigor: data/solucion_poisson_tomic.dat

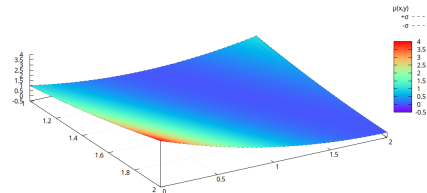
Superficie: Teoría vs Cómputo

Superficie Promedio y Banda de Incertidumbre σ

Vista general (60°, 35°)

Error Absoluto Local: $|T_{Num} - T_{Teoría}|$ 

Vista lateral (25°, 60°)



Conclusiones Generales

1. Paradigma Dominante

- `parallel for` + `reduction` es superior.
- La uniformidad geométrica permite distribución estática ideal.
- **Speedup máx:** $32,5\times$ (64 hilos, malla 512^2).

2. Costo de Sincronización

- Hardware (`atomic`): ~ 6.6 s
- Software (`critical`): ~ 7.1 s
- Preferir siempre `atomic`.

3. Falla del Modelo de Tareas

- El *runtime* colapsa encolando 1,024 tareas/iteración.
- Rendimiento $117\times$ peor que `parallel for`.
- Usar solo para grafos o recursividad.

4. Validez Numérica

- La paralelización **no afecta la física**.
- 100 % de consistencia en el esquema de diferencias finitas iterativo.

Veredicto final: `parallel for` con `reduction` es la solución óptima para EDPs.

Camilo Huertas · Sebastián Rodríguez

Programa Académico de Física
Universidad Distrital Francisco José de Caldas

Docente: Carlos Andrés Gómez Vasco | Sistemas Distribuidos